



Getting Started with Research Computing

From Command Line to Job Scheduling

Website: www.rc.colorado.edu

Documentation: curc.readthedocs.io

Support request form: colorado.service-now.com/req_portal?id=ucb_sc_rc_form



Meet the User Support Team



Layla
Freeborn



Brandon
Reyes



Andy
Monaghan



Michael
Schneider



John
Reiland



Dylan
Gottlieb

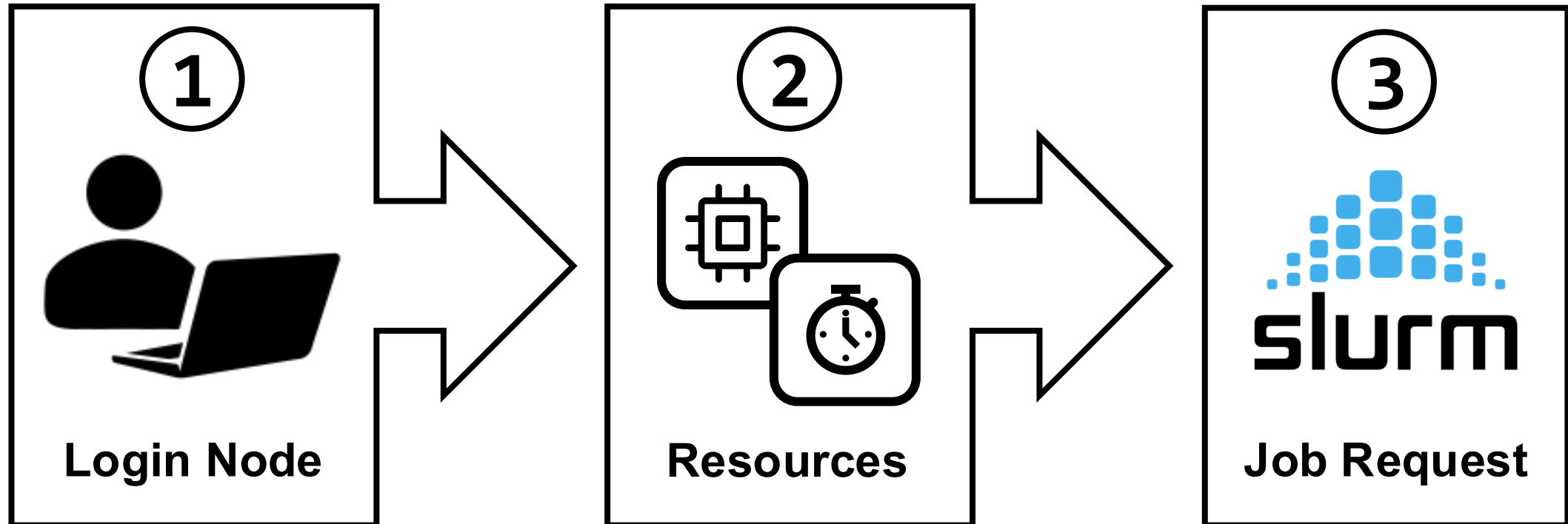


Mohal
Khandelwal



Ask Questions

General Overview of Job Submission



Login Nodes

A Login Node is the entryway onto our system. They are meant for:

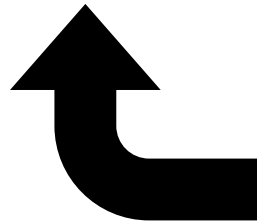
- Viewing or editing files
- Submitting Jobs

Login nodes should **never** be used to run code or software.

Once you are on a terminal, you will know you are on a login node by looking at the command prompt. If it contains “login”, you are on a login node

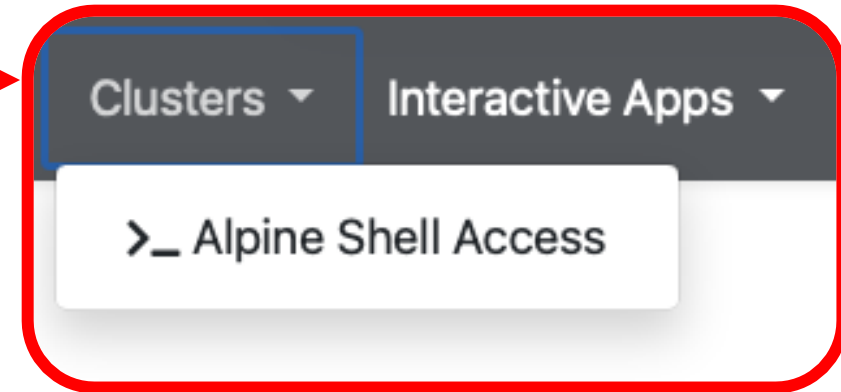
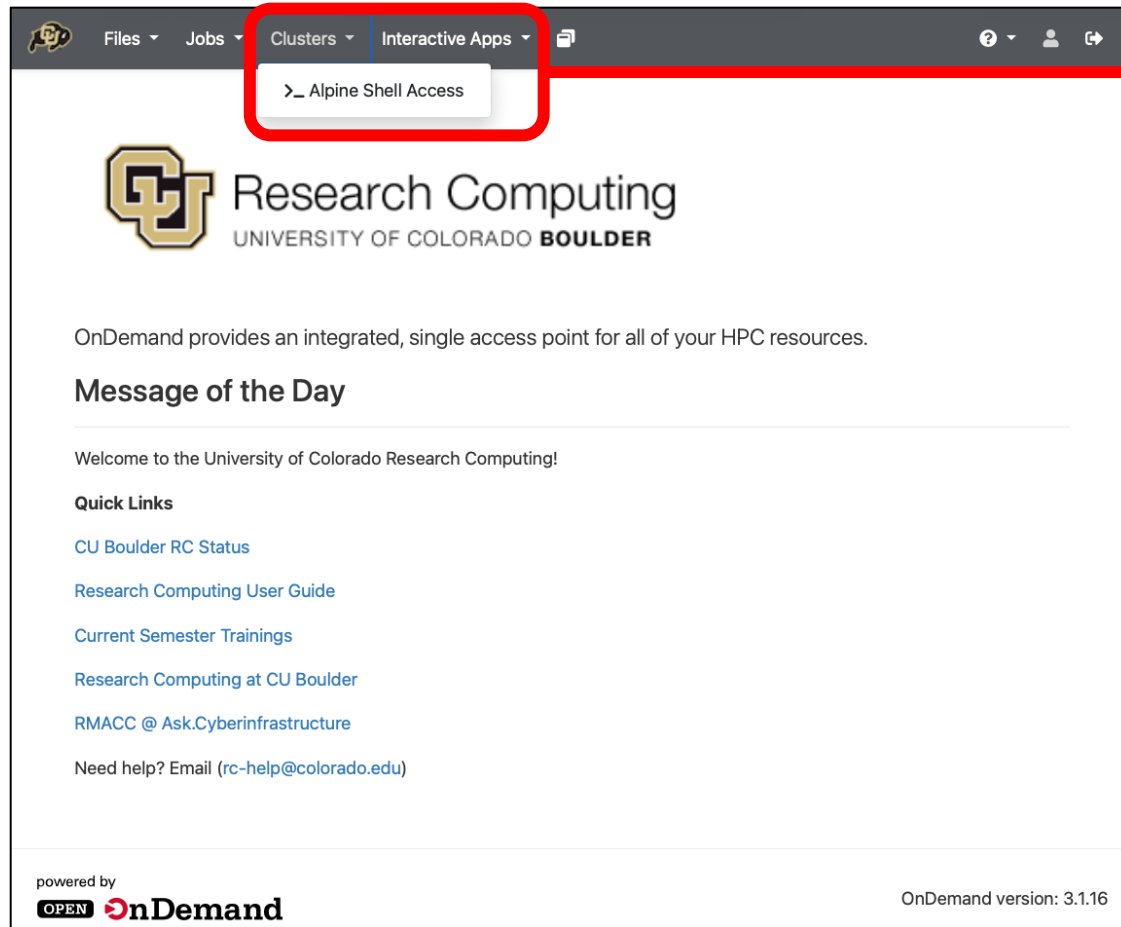
```
[ralphie@login-ci3]$
```

Accessing the Terminal



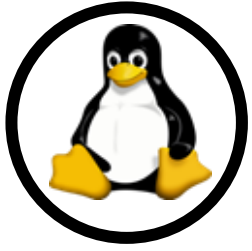
**Available to CU Boulder, CSU,
CU Anschutz**

Accessing the Terminal Cont.

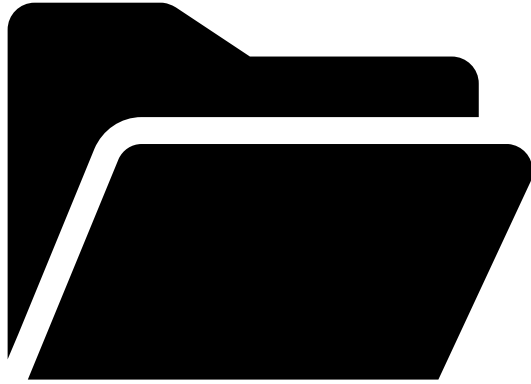


If you would like to access a terminal via ssh,
please see:

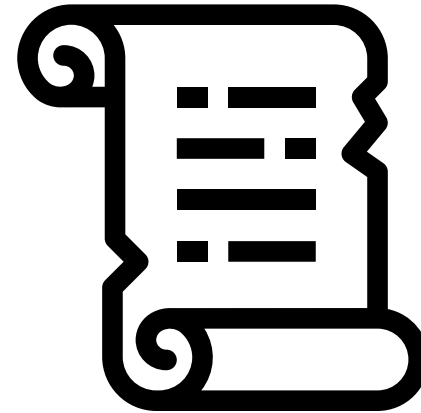
curc.readthedocs.io/en/latest/getting_started/logging-in.html#terminal-access-for-different-operating-systems



Linux Command Line



File Operations



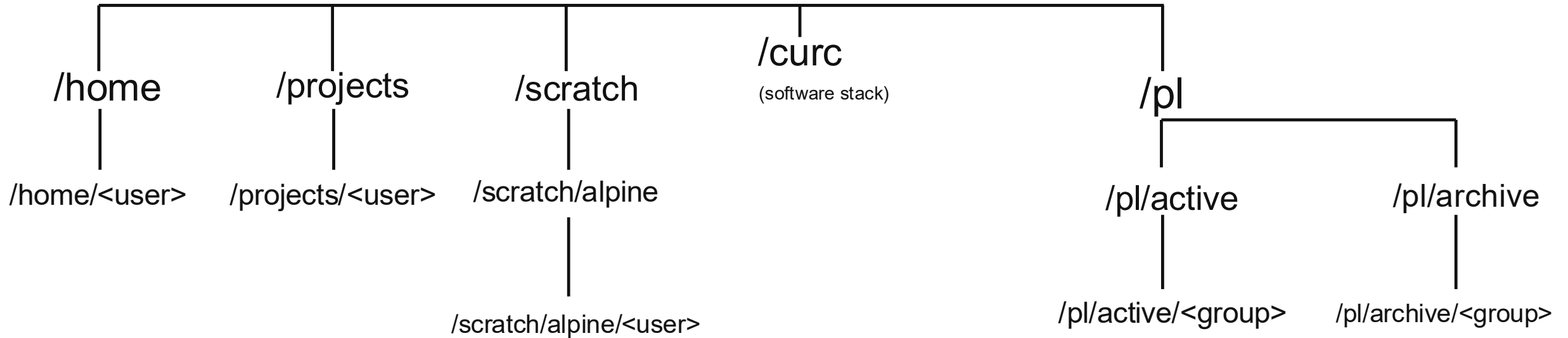
Scripting

List Files and Directories

`ls [option] [file/directory]`

- `-l` (long format)
- `-h` (readable file size)
- `-S` (sort by file size)
- `-t` (sort by modification time)
- `-a` (all files)
- `-r` (reverse sort)

Standard CURC Directories



File Operations

Change Directory

`cd [-o] <path>`

Make Directory

`mkdir [-o] <path>`

Copy / Move Files & Directories

`cp [-o] <source> <dest>`

Delete Files

`rm [-o] <path>`

Create and edit a file – Terminal Text Editors



Vim

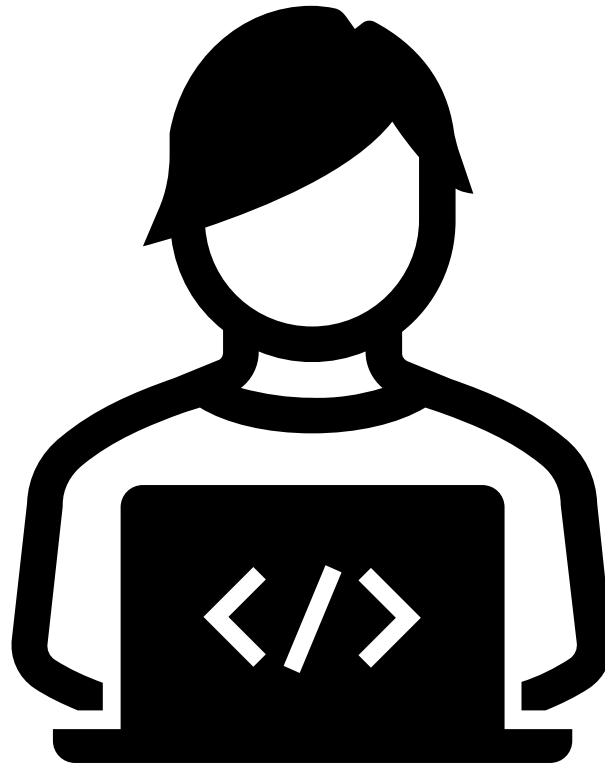
```
      :::  
      iLE88Dj.  :jD88888Dj:  
      .LGitE888D.f8GjjjL8888E;  
      iE  :8888Et.      .G8888.  
      ;i   E888,      ,8888,  
           D888,      :8888:  
           D888,      :8888:  
           D888,      :8888:  
           D888,      :8888:  
           888W,      :8888:  
           W88W,      :8888:  
           W88W:      :8888:  
           DGGD:      :8888:  
                        :8888:  
                        :W888:  
                        :8888:  
                        E888i  
                        tw88D
```

Nano



Emacs

Creating a Bash Script



Shell script basics

- In shell scripts, the first line must specify the shell e.g. `#!/bin/bash`
- The program loader recognizes the `#!` and will interpret the rest of the line (`/bin/bash`) as the interpreter program.
- If a line starts with `#`, it is a comment and is not run.

```
#!/bin/bash
```

```
#look at the files in /tmp
```

```
cd /tmp
```

```
ls
```

Shell to run

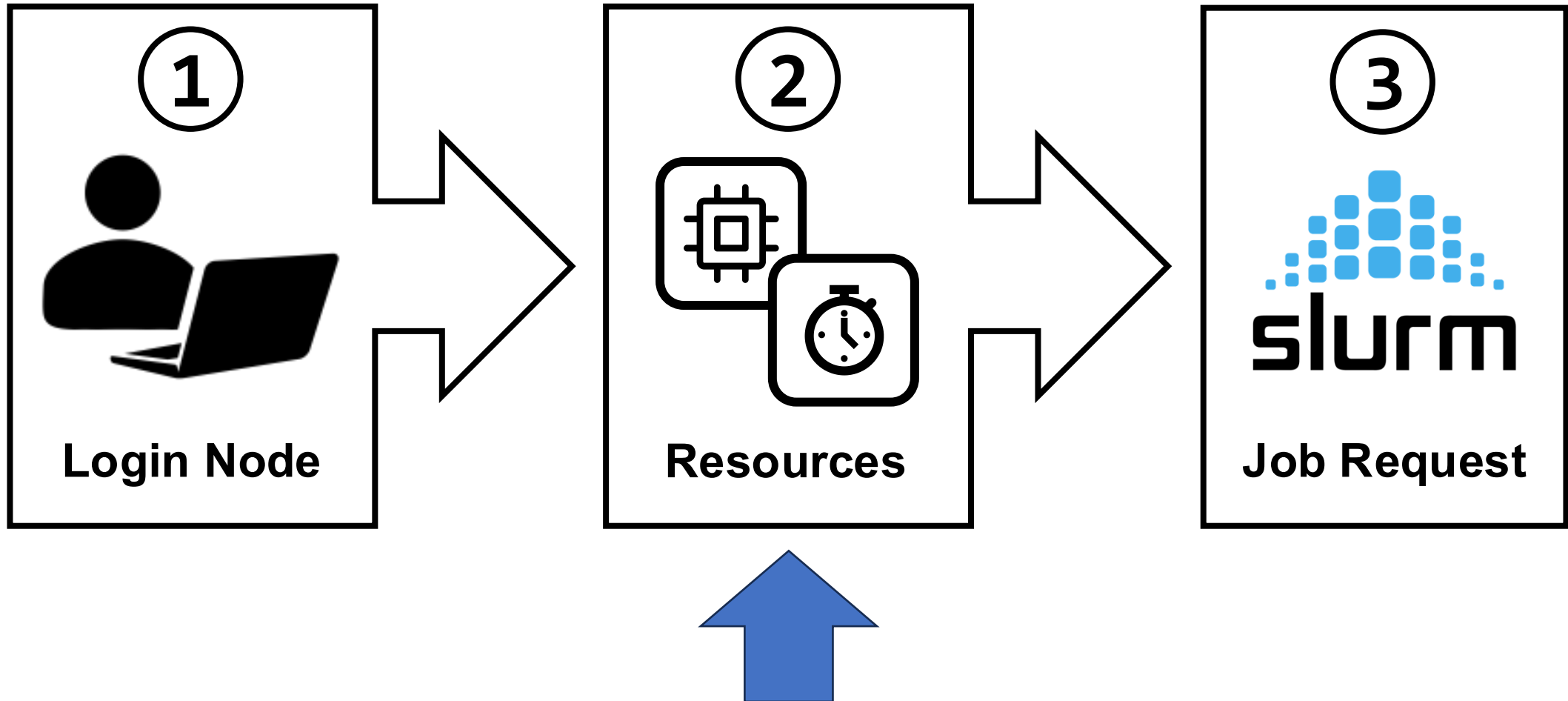
Comment

Change directories

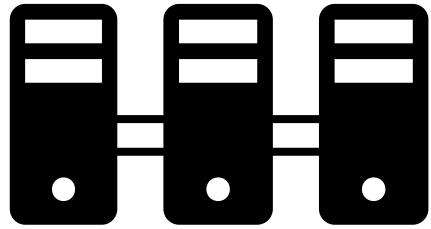
List everything in /tmp



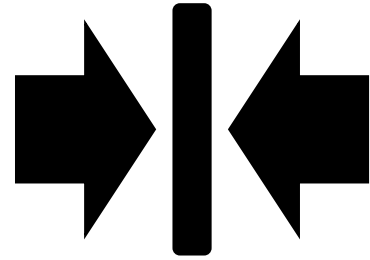
Selecting Resources



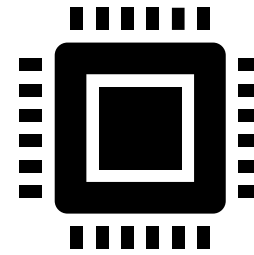
Common Resource Requests



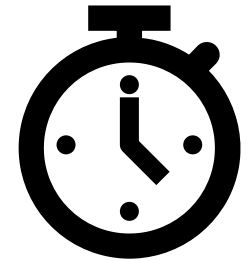
Partition



**Quality of
Service**



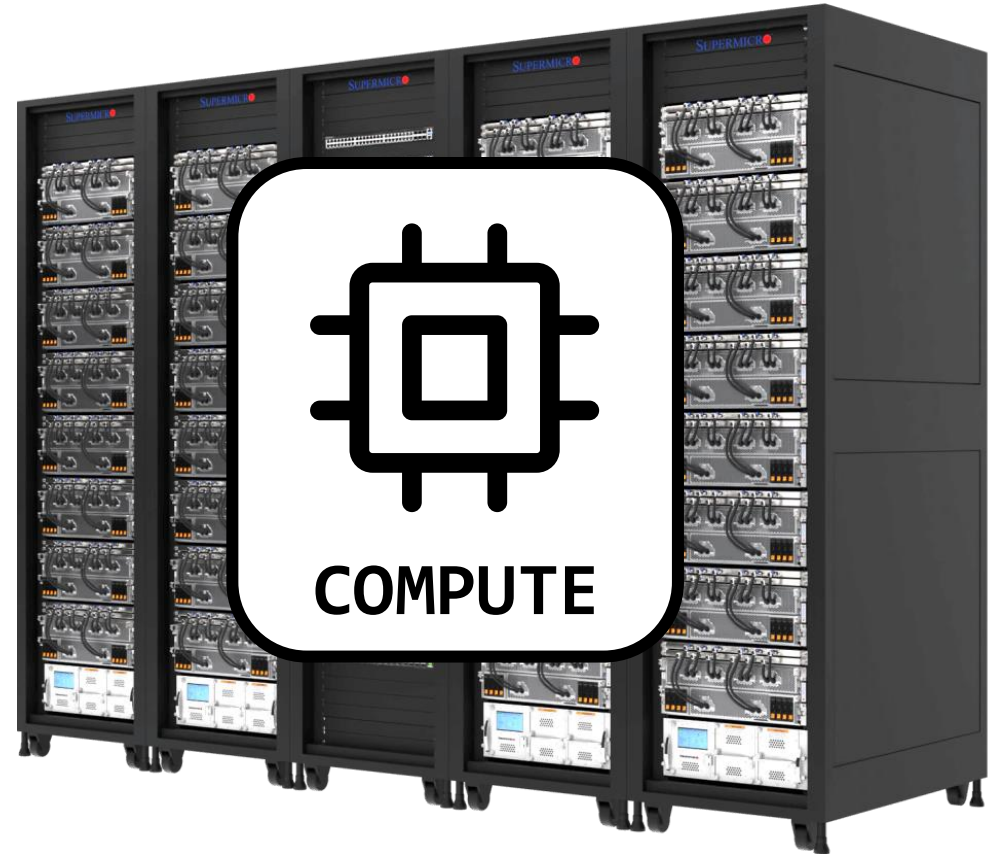
**Cores, GPUs,
and Nodes**



Wall Time

Alpine Partitions

- Partitions are a collection of compute nodes with common characteristics
- Compute nodes can be thought of as a desktop without a monitor
- For in-depth information on partitions see:
curc.readthedocs.io/en/latest/clusters/alpine/alpine-hardware.html#partitions



Alpine Partitions - acpu

acpu
General CPU



Alpine Partitions - amem

acpu

General CPU

amem

High Memory
CPU



Alpine Partitions – NVIDIA GPUs

acpu

General CPU

amem

High Memory
CPU



aa100

al40

ah200

artxpro6000

Nvidia GPUs



Alpine Partitions – AMD GPUs

acpu

General CPU

amem

High Memory
CPU



aa100

al40

ah200

artxpro6000

Nvidia GPUs

ami100

AMD GPUs

Alpine Testing Resources

- For several of the different types of hardware, we provide testing resources. These resources enable quicker access.
 - There are restrictions on the amount of resources you can request
- Access to testing resources is enabled through a combination of partition and QoS selection
- Available testing resources:
 - Standard CPUs
 - Compute nodes for verifying parallel applications
 - NVIDIA A100 and AMD MI100 GPUs

For more information see: curc.readthedocs.io/en/latest/clusters/alpine/alpine-hardware.html#special-purpose-resources

Quality of Service (QoS)

- A QoS enables us to specify additional constraints for a given partition
- On Alpine, a QoS can be used to run:
 - Long jobs (> 24 hours)
 - Specify testing/compile resources
- A QoS can also set limits on the number of active jobs and resources consumed, which enables us to
 - Provide more resources to 24-hour jobs and less to longer jobs
 - Limits the number of Jobs per user to provide a more equitable system

Quality of Service (QoS) Examples

QoS	Description	Max wall time	Max jobs/user	Max hardware/user
cpu-normal	24-hour QoS for acpu	24 hours	1000	128 nodes
cpu-long	For acpu jobs needing longer wall times	7 days	200	20 nodes
gpu-normal	24-hour QoS for GPU partitions	24 hours	1000	Constraints set per GPU type
gpu-long	For GPU jobs needing longer wait times	7 days	200	Constraints set per GPU type
compile	For jobs submitted to the acompile partition	12 H	4	8 CPUs

For all available QoS, see curc.readthedocs.io/en/latest/clusters/alpine/alpine-hardware.html#quality-of-service-qos

Selecting Hardware

- Once a partition and QoS are selected, we need to say how much of the hardware we would like to use
- The most common selections are
 - The number of cores (often called “ntasks”)
 - On our system the default behavior is each task corresponds to 1 core
 - The number computes nodes (often referred to as “nodes”)
 - The type of GPU

GRES Type

- On Alpine we utilize general resources (GRES) to select the GPU type you would like within a given GPU partition
- Within several of our GPU partitions, we have GPUs with different amounts of Video Random Access Memory (VRAM)
 - VRAM is the amount of memory a GPU has for calculations
- Multi-Instance GPU (MIG) allows us to split up full GPUs with respect to their compute power and memory
 - MIG is fantastic and works flawlessly with most code
 - MIG limitations: docs.nvidia.com/datacenter/tesla/mig-user-guide/latest/deployment-considerations.html#application-considerations

GRES Type Examples

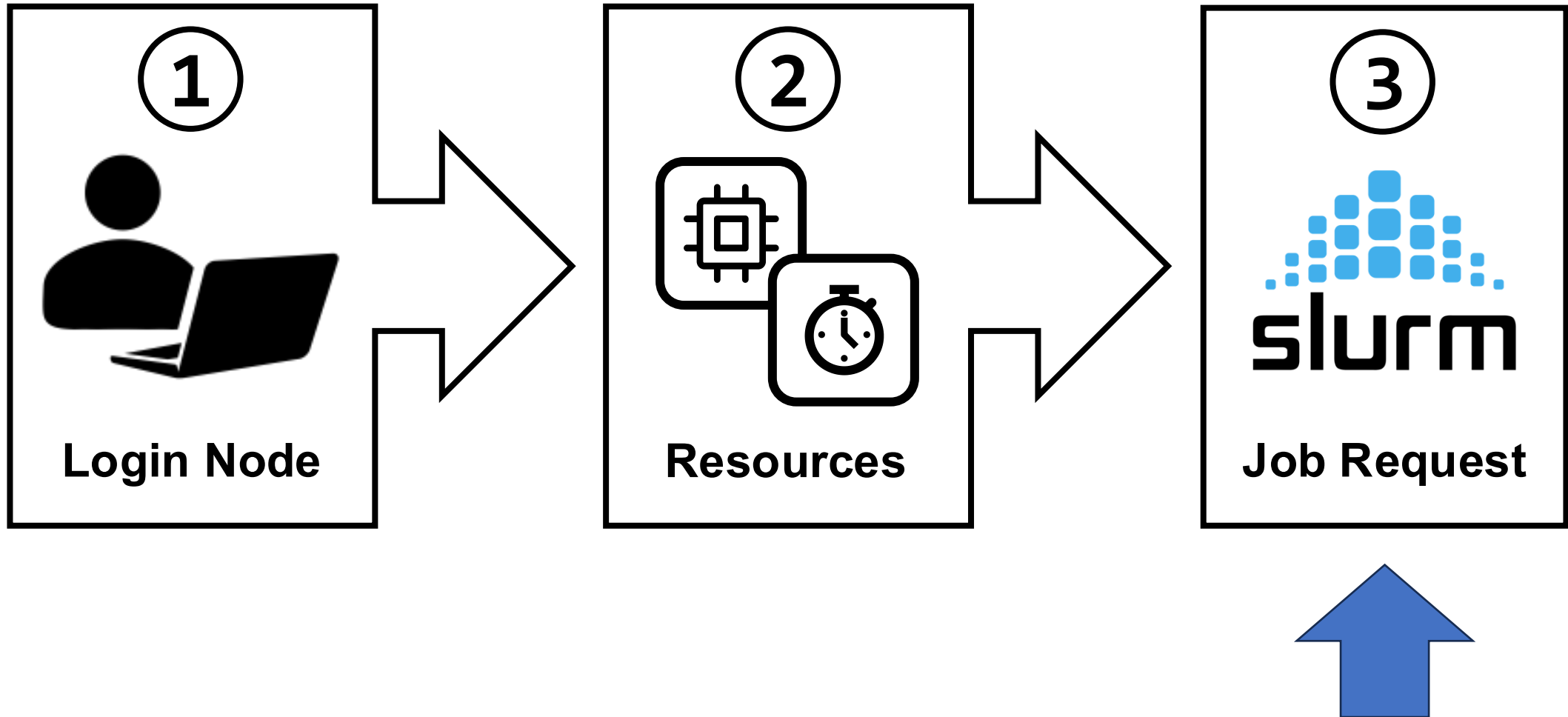
GRES type	Description
rtx_pro_6000	A full NVIDIA RTX Pro 6000 with 96 GB of VRAM
rtx_pro_6000_2g.48gb	An NVIDIA RTX Pro 6000 with 48 GB of VRAM made possible by MIG
rtx_pro_6000_1g.24gb	An NVIDIA RTX Pro 6000 with 24 GB of VRAM made possible by MIG
a100-40gb	A full NVIDIA A100 GPU with 40 GB of VRAM
h200	A full NVIDIA H200 with 141 GB of VRAM
mi100	A full AMD MI100 GPU with 34 GB of VRAM

For a full list of GRES types and associated constraints, see
curc.readthedocs.io/en/latest/clusters/alpine/alpine-hardware.html#general-resources-gres

Walltime

- Walltime or “run time” is how long you want to run on the selected resources
- How long do I specify?
 - We often recommend people do a smaller version of their workflow to get a sense of the runtime and then scale it up
- Downsides to requesting too much time
 - Increase in wait times
 - If you do not use the resources, you are making the Slurm algorithm inefficient, which can impact overall system utilization

Requesting a Job



Planned Maintenance

On every first Wednesday of the month (like today!) we perform a planned maintenance (PM). During this PM we do not allow you to run jobs on the system. Additionally, getting on the system may not be possible.

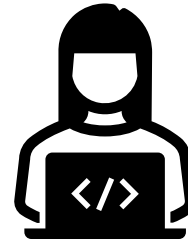
Common items completed during PMs:

- Patching and updating the system
- Improving the efficiency of the system
- Rolling out system configuration changes
- Introduction of new features

Batch vs Interactive Jobs



**Batch
Job**



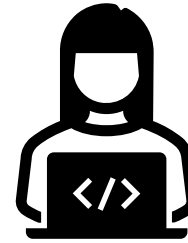
**Interactive
Job**

Batch Jobs

- Runs a job script
- No need for user input
- Best for tested workflows and long/big jobs
- Runs when resources are available



**Batch
Job**

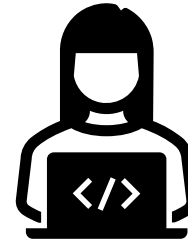


**Interactive
Job**

Interactive Jobs



**Batch
Job**



**Interactive
Job**

- Terminal or GUI
- Requires active user input
- Best for development and exploration

Slurm Directives

Slurm is a popular resource manager for High-Performance Computing systems it

- Grabs the resources you are requesting
- Determines when your job will run using an equitable algorithm based on user and system usage

Slurm directives are how we tell Slurm what resources we would like

- Used for both interactive and batch jobs

Common Slurm Directives

- ★ • **Partition**
 - `--partition=<partition_name>`
- ★ • **Quality of service (QoS)**
 - `--qos=<qos>`
- ★ • **Number of nodes to run on**
 - `--nodes=<nodes>`
- ★ • **Number of cores to run on**
 - `--ntasks=<number-of-tasks>`
- ★ • **Wall time**
 - `--time=<wall time>`
- ★ • **Number of GPUs and GRES type (“N” is # of GPUs)**
 - `--gres=gpu:<GRES type>:N`

★ - Must be specified for every job

★ - Must be specified for certain jobs

Common Slurm Directives cont.

- **Job name**
 - `--job-name=<jobname>`
- **Output – Where all output that would be written to the terminal should go**
 - `--output=<name>`
- **Allocation – Account to “charge to”**
 - `--account=<account_name>`
- **Send an email when events happen in the job**
 - `--mail-type=<type>`
- **Email address to send updates to**
 - `--mail-user=<user>`



Starting an interactive job

acompile

- A custom CURC command
- Submits an interactive job to the acompile testing resources
- Limit of 4 CPUs for up to 12 hours
- Focused on compiling/building software and small workflows

sinteractive

- Request any partition, more resources, and longer walltime
- Subject to standard wait times (not immediate access)
- As the name implies, allows you to access a job interactively

Running an interactive job

Here we will run a Python script using an interactive job

1. First request resources:

```
[ralphie@login-ci3]$ sinteractive --partition=acpu --qos=cpu-normal --time=10 --ntasks=1 --nodes=1
```

2. When the job starts you will be put on a compute node and you can execute your commands:

```
[ralphie@c3cpu-e2-u1]$ module load anaconda  
[ralphie@c3cpu-e2-u1]$ python my_very_cool_script.py
```

3. To quit:

```
[ralphie@c3cpu-e2-u1]$ exit
```

Submitting a Batch Job

- Batch Jobs are jobs are run by submitting a job script to the scheduler
- A job script is simply a script that includes **SLURM directives** (resource specifics) ahead of any commands.

Anatomy of a Job Script

```
#!/bin/bash
```

```
## Directives
```

```
#SBATCH --<option>=<value>
```

```
## Software
```

```
module load <software>
```

```
## User scripting
```

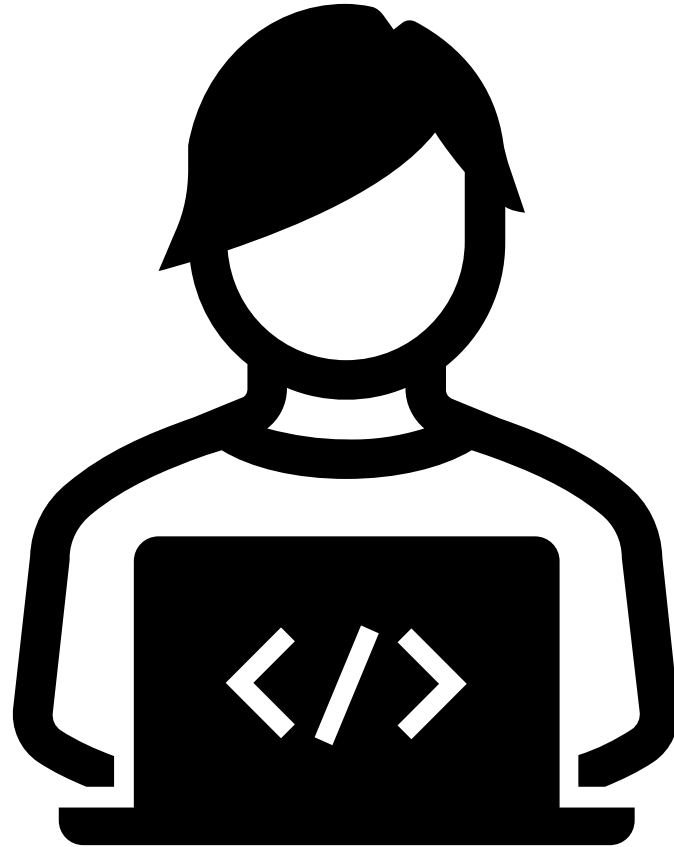
```
<command>
```



Directives in a Job Script

`#SBATCH --<option>=<value>`

Creating a Job Script



Example job script

```
#!/bin/bash

## Directives
#SBATCH --ntasks=1           # Number of requested tasks/cores
#SBATCH --time=0:01:00       # Max run time
#SBATCH --partition=acpu     # Specify Alpine CPU node
#SBATCH --qos=cpu-normal     # Specify QoS
#SBATCH --output=test_%j.out # Rename standard output file

## Software
module purge                 # Purge all existing modules

## User commands
echo "This is a test submitted by $USER"
```

Submitting a Job script

`sbatch <job_file> <other-directives>`

- Command to submit a job to the Slurm job scheduler
- If we created the job script “my_first_job.sh” then we would submit it as follows:

`sbatch /path/to/my_first_job.sh`

- Modify Slurm directives in the batch script:

`sbatch /path/to/my_first_job.sh --ntasks=12`

Checking your jobs

- **squeue**: monitor your jobs status **in queue and while running**:
 - By default, shows all jobs in queue, can specify using:

```
$ squeue -u <username>  
$ squeue -p <partition>
```

- **sacct**: check back on usage statistics of **previous Jobs**
 - By default, only checks all jobs from the start of the current day can specify using:

```
$ sacct -u <username>  
$ sacct --start=MM/DD/YY -u <username>  
$ sacct -j <job-id>
```

Checking your jobs

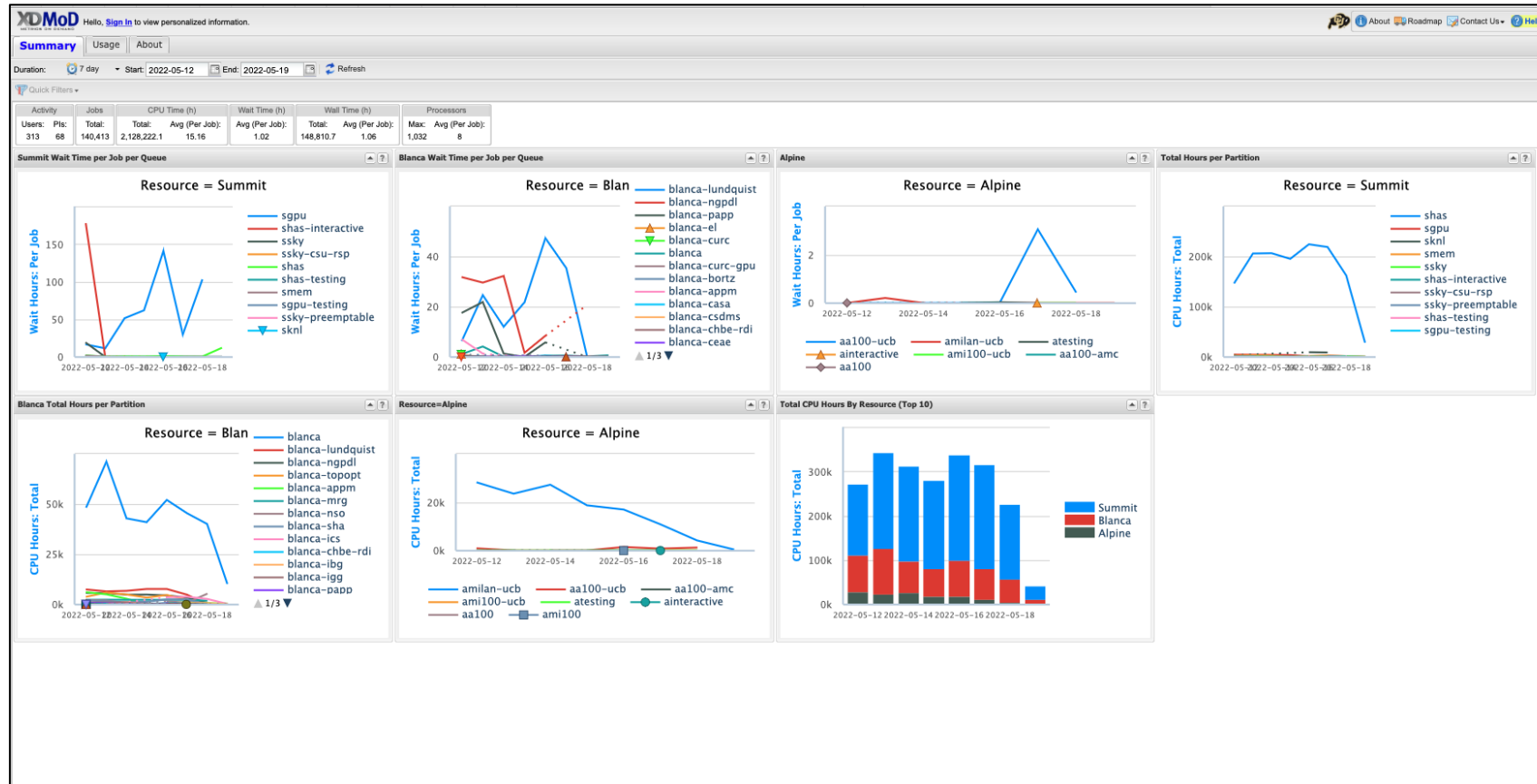
- To check the percentage of CPU, GPU, and memory usage of a job **after it completes**, use `seff`

```
$ module load slurmttools  
$ seff <job number>
```

- To see general statistics for your submitted jobs, use `jobstats`

```
$ module load slurmttools  
$ jobstats <username> <# of days>
```

Job Monitoring - XDMOD



Help! I'm stuck, where do I go?

- **Documentation**: curc.readthedocs.io
- **Trainings with Center for Research Data and Digital Scholarship (CRDDS)**:
www.colorado.edu/crdds
- **Support request form**: colorado.service-now.com/req_portal?id=ucb_sc_rc_form
- **Consult Hours** (Tuesday 1:00-2:00pm in-person or virtually, Thursday 1:00-2:00pm virtually)

Survey and feedback

<https://s.zoom.us/j/bPITUoGhy>

